

EuroMobilityDataHub

Plateforme centralisée de données ferroviaires européennes

BLOC 1 — Collecter, transformer et sécuriser des données

Certification RNCP39586 — Ingénieur en science des données

Candidat : Ivan Mustiere

Spécialisation : M2 Data Engineer — YNOV

Date : 22/06/2026

Sommaire

Partie 1 — Stratégie de collecte des données

- 1.1 - Objectifs de la collecte
- 1.2 - Données utiles et nécessaires
- 1.3 - Sources de données identifiées
- 1.4 - Moyens envisagés pour la collecte
- 1.5 - Exemples de collecte de données
- 1.6 - Synthèse de la stratégie de collecte

Partie 2 — Automatisation de la collecte

- 2.1 - Architecture d'automatisation
- 2.2 - Orchestration batch avec Apache Airflow
- 2.3 - Collecte temps réel avec Apache Kafka
- 2.4 - Garantie de la fraîcheur des données
- 2.5 - Synthèse de la stratégie d'automatisation

Partie 3 — Stockage des données

- 3.1 - Stratégie de stockage — Architecture Medallion
- 3.2 - Modèle de données
- 3.3 - PostgreSQL — Base opérationnelle (Silver temps réel)
- 3.4 - Amazon S3 et Snowflake — Du Bronze au Gold
- 3.5 - Synthèse — Flux de données bout en bout

Partie 4 — Transformation des données et processus ETL

- 4.1 - Sélection des technologies de traitement
- 4.2 - Transformations appliquées aux données
- 4.3 - Processus ETL complet
- 4.4 - Résultats des transformations
- 4.5 - Synthèse

Partie 5 — Sécurité des données

- 5.1 - Politique de sécurité des données
- 5.2 - Architecture de sécurité
- 5.3 - Synthèse de la politique de sécurité

Conclusion

Annexes

Introduction

Malgré l'espace Schengen, l'Europe souffre encore d'une forte fragmentation de ses données. Il existe plus d'une vingtaine d'opérateurs ferroviaires nationaux, tels que la SNCF, Deutsche Bahn, Trenitalia, Renfe, ÖBB ou encore Eurostar. Cependant, il n'existe actuellement aucun véritable système centralisé. Chacun de ces acteurs dispose de ses propres systèmes d'information, formats de données et interfaces techniques, rendant toute interopérabilité complexe.

Des solutions commerciales comme Trainline, Omio ou Rome2Rio tentent de répondre à ce besoin d'agrégation, mais elles restent des solutions propriétaires : leurs données ne sont pas librement accessibles et leur couverture des réseaux urbains demeure partielle.

EuroMobilityDataHub tente de combler ce manque. Le projet vise à concevoir une architecture de données open source capable de collecter, stocker et exposer l'ensemble des informations ferroviaires sous une forme unifiée et exploitable par des tiers. Ce dossier se concentrera principalement sur les stratégies de collecte, de transformation et de sécurisation des données mises en place dans le cadre du projet.

Partie 1 — Stratégie de collecte des données

1.1 Objectifs de la collecte

La collecte de données étant le point de départ de toute architecture data, elle devra répondre à trois objectifs principaux dans le cadre du projet EuroMobilityDataHub :

- Centraliser les informations ferroviaires européennes issues de sources hétérogènes
- Garantir la fraîcheur des données en intégrant des flux temps réel
- Respecter le cadre réglementaire européen (RGPD et les conditions d'utilisation API)

Ces objectifs guident les choix des sources, des techniques de collecte et de la fréquence d'actualisation des données

1.2 Données utiles et nécessaires

Quatre catégories de données ont été identifiées comme obligatoire au fonctionnement du projet :

Catégorie	Description
Horaires et itinéraires	Départs, arrivées, correspondances, durées de trajet pour chaque opérateur.
Tarifs	Prix des billets par classe, type de billet (flexible, non remboursable), promotions en cours.
Perturbations temps réel	Retards, suppressions de trains, informations de substitution.
Référentiel géographique	Données statiques : gares, coordonnées GPS, pays, nom local et nom normalisé.

Tableau 1 — Catégories de données nécessaires au projet

1.3 Sources de données identifiées

L'écosystème de données ferroviaires européen repose sur trois grands types de sources

a) APIs officielles des opérateurs

Plusieurs opérateurs ferroviaires exposent des APIs publiques ou semi-publiques permettant d'accéder à leurs données :

- SNCF Connect API (France) - horaires, itinéraires, perturbations en temps réel
- Deutsche Bahn Open API (Allemagne) - horaires et données de gares
- NS API (Pays-Bas) - horaires et perturbations du réseau néerlandais
- SNCB API (Belgique) - horaires et correspondances
- Eurostar API - horaires et disponibilités sur les liaisons transmanche

b) Flux GTFS (General Transit Feed Specification)

Un autre format qui s'est imposé comme un standard dans la publication des données de transport en Europe est le GTFS. Il existe sous deux variantes principales. GTFS Static regroupe, dans des fichiers CSV compressés, les horaires planifiés, les arrêts et les itinéraires. Les opérateurs mettent généralement ces données à jour chaque semaine ou chaque mois. GTFS-RT (Real Time) permet quant à lui de diffuser en continu des flux au format Protobuf contenant des informations en temps réel : mises à jour d'horaires, alertes voyageurs et positions des véhicules.

Concernant la disponibilité des données, des flux GTFS sont proposés par plusieurs opérateurs européens, notamment Renfe (Espagne), Trenitalia (Italie), ÖBB (Autriche), ainsi que par la majorité des réseaux régionaux. Ces flux sont notamment référencés sur la plateforme Mobility Database.

c) Sites web des opérateurs (Web Scraping)

Certains opérateurs ne disposent pas d'API publique pour leurs tarifs, notamment Renfe et Trenitalia pour les prix dynamiques. Dans ces cas, une collecte par web scraping est réalisée sur les sites officiels, dans le strict respect des conditions générales d'utilisation et du cadre légal européen.

1.4 Moyens envisagés pour la collecte

Quatre techniques de collecte sont déployées en complémentarité pour couvrir l'ensemble des sources identifiées :

Technique	Source concernée	Données collectées
Appels API REST	SNCF, DB, NS, SNCB, Eurostar	Horaires, perturbations, disponibilités
GTFS Static / RT	Renfe, Trenitalia, ÖBB, réseaux régionaux	Horaires planifiés, temps réel
Web Scraping	Renfe, Trenitalia (prix)	Tarifs dynamiques, promotions

Tableau 2 — Techniques de collecte selon la source

À ces techniques de collecte s'ajoute l'utilisation de Requêtes SQL sur notre *base de staging interne* afin de consolider et d'historiser l'ensemble des données récupérées.

1.5 Exemples de collecte de données

Cette section présente concrètement les quatre techniques de collecte mises en œuvre, avec les outils et bibliothèques utilisés.

a) Appel API externe - SNCF Connect API

L'API SNCF Connect est une API REST publique accessible sur inscription. Elle permet d'interroger les horaires et les perturbations en temps réel. L'authentification se fait par clé API passée dans le header de la requête HTTP.

Exemple de requête Python pour récupérer les départs depuis Paris Gare de Lyon :

```
import requests
API_KEY = 'votre_cle_sncf'
url =
'https://api.sncf.com/v1/coverage/sncf/stop_areas/stop_area:SNCF:87686006/departures'
headers = {'Authorization': API_KEY}
response = requests.get(url, headers=headers)
data = response.json()
```

La réponse JSON est analysée afin d'en extraire plusieurs informations : le numéro de train, la destination, l'heure de départ théorique ainsi que l'heure réelle en cas de perturbation. Ces données sont ensuite normalisées vers le modèle pivot d'EuroMobilityDataHub, basé sur le standard Transmodel

Qualité de la collecte : l'exhaustivité est garantie par la pagination de l'API (paramètre `start_page`). L'exactitude est vérifiée par comparaison croisée avec les flux GTFS-RT. Le droit d'utilisation est encadré par les CGU de l'API SNCF (open data, usage non-commercial autorisé).

b) Flux GTFS-RT — Données temps réel de la Renfe / ÖBB

Le format GTFS-RT permet de recevoir en continu les positions des trains et les retards sous forme de flux binaires standardisés (Protocol Buffers). En Python, la bibliothèque `gtfs-realtime-bindings` est utilisée pour décoder ces flux. (Annexe 1)

Les données ainsi décodées permettent d'obtenir une vue globale et standardisée du réseau ouest-européen sans dépendre d'API REST propriétaires.

c) Web Scraping - Tarifs Renfe (Espagne)

Certaines entreprises, comme Renfe, ne proposent pas d'API publique pour leurs tarifs. La seule manière d'accéder à ces données de manière automatisée est de réaliser du web scraping sur le site officiel de Renfe. Pour cela, des bibliothèques comme Playwright en Python seront utilisées afin de gérer le rendu dynamique des pages. (Annexe 2)

Le scraping sera effectué dans le respect des conditions d'utilisation des sites (`robots.txt` consulté, fréquence de requêtes limitée, etc.). Les prix collectés sont utilisés pour conserver un historique tarifaire.

d) Requête SQL - Consolidation

Une fois les flux API, GTFS et scraping rapatriés dans la base de staging PostgreSQL, des requêtes SQL prennent le relais pour consolider l'ensemble : recouper les enregistrements, repérer les doublons d'un opérateur à l'autre et ne conserver, pour chaque trajet, que la version la plus récente. (Annexe 3)

Côté qualité, l'exhaustivité tient au balayage de toutes les sources chargées sur les dernières 24 heures, et l'exactitude à la déduplication qui privilégie la donnée la plus fraîche.

e) Web Crawling - Découverte de nouvelles gares

Là où le scraping extrait des données d'une page connue, le crawling parcourt automatiquement l'arborescence d'un site pour y découvrir de nouvelles ressources. Chaque mois, un crawler explore ainsi les pages « gares » des opérateurs afin de repérer les nouvelles stations à intégrer au référentiel. (Annexe 4)

Côté qualité, l'exhaustivité repose sur le parcours récursif de tous les liens internes jusqu'à épuisement de la file à visiter, et l'exactitude sur la comparaison avec le référentiel existant, qui ne retient que les gares réellement nouvelles. Le cadre réglementaire est respecté via la consultation du fichier robots.txt, une fréquence de crawl limitée et l'absence de toute donnée personnelle.

1.6 Synthèse de la stratégie de collecte

La stratégie de collecte définie pour EuroMobilityDataHub combine trois techniques complémentaires pour couvrir l'ensemble du réseau ferroviaire européen. Chaque technique a été choisie en fonction de la source et du type de données visées. L'ensemble repose sur des outils open source éprouvés (Python, Playwright, Requests) et s'inscrit dans le respect du cadre réglementaire européen (RGPD, conditions d'utilisation des APIs et sites web concernés).

Partie 2 — Automatisation de la collecte

La collecte manuelle de données n'est pas envisageable dans un projet de l'envergure d'EuroMobilityDataHub : avec une vingtaine de sources hétérogènes, des millions d'enregistrements d'horaires et des flux temps réel continus, l'automatisation constitue un pilier central de l'architecture. Deux approches complémentaires sont mises en place : un ordonnancement batch via Apache Airflow pour les collectes périodiques, et une pipeline de streaming via Apache Kafka pour les flux en temps réel.

2.1 Architecture d'automatisation

L'architecture d'automatisation repose sur une distinction fondamentale entre deux types de données :

Type de données	Exemples	Outil retenu
Batch (real fixe)	GTFS Static, horaires, tarifs scrapés	Apache Airflow
Streaming (temps réel)	GTFS-RT, retards, alertes perturbations	Apache Kafka

Tableau 3 — Choix des outils d'automatisation selon le type de données

Ces deux outils sont complémentaires et forment ensemble le socle d'automatisation du projet EuroMobilityDataHub. Ils sont déployés via Docker Compose pour garantir la portabilité et la reproductibilité de l'environnement.

2.2 Orchestration batch avec Apache Airflow

a) Présentation de l'outil

Apache Airflow est un orchestrateur de workflows de données. Concrètement, elle possède des pipelines en Python sous forme de DAGs (*Directed Acyclic Graphs*), on programme leur exécution avec des expressions CRON, et on suit leur état depuis une interface web.

Airflow a été retenu pour les raisons suivantes :

- Planification fine : exécution possible à la minute, à l'heure, quotidiennement ou à une fréquence personnalisée
- Gestion des dépendances : Un DAG enchaîne collecte, validation et chargement dans un ordre strict : les dépendances se gèrent nativement
- Relance automatique : Une tâche échoue ou une erreur réseau voire l'API indisponible ? Airflow la retente, selon une politique configurable
- Observabilité : chaque exécution est tracée avec ses logs, son statut et sa durée.

b) DAG de collecte des données GTFS Static

Le DAG suivant orchestre la collecte hebdomadaire des fichiers GTFS Static pour l'ensemble des opérateurs européens. Nous avons décidé arbitrairement que le script s'exécutera tous les lundis à 3h du matin, car l'heure creuse minimise l'impact. (Annexe 5)

c) DAG de collecte des tarifs par Web Scraping

Un second DAG gèrera également la collecte quotidienne des tarifs via web scraping. Il s'exécutera à 6h du matin pour récupérer les prix du jour et les charger ensuite dans la base PostgreSQL.

```
with DAG(
    dag_id='collect_fares_scraping',
    schedule_interval='0 6 * * *',
    start_date=datetime(2025, 9, 1),
    catchup=False,
) as dag:

    scrape = PythonOperator(
        task_id='scrape_renfe_fares',
        python_callable=scrape_renfe,
    )

    load = PythonOperator(
        task_id='load_to_staging',
        python_callable=load_fares_to_postgres,
    )

    scrape >> load
```

2.3 Collecte temps réel avec Apache Kafka

a) Présentation de l'outil

Apache Kafka est un outil de streaming distribuée. Dans EuroMobilityDataHub, elle consomme les flux GTFS-RT que les opérateurs diffusent pour mettre à jour les horaires en temps réel et alerter des perturbations.

Trois raisons ont motivé ce choix :

- Kafka a la capacité de pouvoir traiter plusieurs millions de demande par seconde, avec une latence généralement sous la seconde, ce qui est suffisant pour notre cas d'usage
- Sa durabilité : les messages sont persistés sur disque, permettant la relecture en cas de défaillance d'un consommateur
- Le découplage producteur/consommateur permet surtout d'éviter que les scrapers soient bloqués si les traitements en aval ralentissent

b) Architecture de la pipeline temps réel

La pipeline temps réel se décompose en trois étapes :

- Un script Python interroge le flux GTFS-RT toutes les 30 secondes (on a choisi 30s car descendre plus basse surcharge inutilement les API des opérateurs)
- Topic Kafka : les messages sont conservés pendant 24h, ce qui permet de rejouer les données si besoin et de faire des analyses posteriori
- Consommateur : un service Python tournera en continu, lit le topic, corrige les messages et met à jour la base PostgreSQL avec les horaires de départ réels

Exemple :


```

from kafka import KafkaProducer
from google.transit import gtfs_realtime_pb2
import requests, time

producer = KafkaProducer(bootstrap_servers='kafka:9092')

def stream_sncf_realtime():
    url = 'https://proxy.transport.data.gouv.fr/resource/sncf-gtfs-rt-trip-updates'
    while True:
        response = requests.get(url)
        feed = gtfs_realtime_pb2.FeedMessage()
        feed.ParseFromString(response.content)
        for entity in feed.entity:
            producer.send('sncf-realtime', entity.SerializeToString())
        producer.flush()
        time.sleep(30)

stream_sncf_realtime()

```

2.4 Garantie de la fraîcheur des données

L'automatisation mise en place garantit la qualité des données à deux niveaux :

Donnée	Fréquence de mise à jour	Mécanisme
Horaires GTFS Static	Hebdomadaire (lundi 2h)	DAG Airflow avec détection de version (hash MD5)
Tarifs (scraping)	Quotidienne (6h)	DAG Airflow + horodatage de chaque enregistrement
Retards et perturbations	Toutes les minutes	Pipeline Kafka (GTFS-RT Protobuf)
Nouvelles gares (crawling)	Mensuelle (1er du mois)	DAG Airflow + comparaison avec référentiel existant

Tableau 4 — Fréquence de mise à jour et mécanismes par type de donnée

En complément, un mécanisme de surveillance est mis en place. Une alerte se déclenche si aucune donnée n'a été reçue d'une source depuis un délai anormalement long. Cette alerte sera envoyée par email et le DAG Airflow sera marqué en échec, déclenchant ainsi une relance automatique. Un dashboard Grafana connecté à Prometheus supervise en temps réel l'état de chaque pipeline de collecte : la fraîcheur des données par opérateur, le nombre de messages Kafka traités ainsi que le statut des DAGs Airflow.

2.5 Synthèse de la stratégie d'automatisation

Nous utiliserons Apache Airflow pour l'orchestration batch et Apache Kafka pour le streaming en temps réel. Cela nous offre une couverture totale des besoins d'automatisation du projet EuroMobilityDataHub. Toutes les données collectées le seront à une fréquence adaptée à leur nature.

L'ensemble des workflows sera versionné sur Git et déployé via Docker Compose, ce qui garantit la reproductibilité de l'environnement et facilite la maintenance.

Partie 3 — Stockage des données

Une fois collectées et automatisées, les données doivent être stockées de manière structurée, performante et évolutive. Le volume de données d'EuroMobilityDataHub est significatif : plusieurs millions d'horaires ferroviaires, des dizaines de millions de mises à jour temps réel par mois via GTFS-RT, et des tarifs actualisés quotidiennement pour l'ensemble du réseau européen. La stratégie de stockage doit concilier performance d'accès, scalabilité et intégrité des données à travers trois couches distinctes.

3.1 Stratégie de stockage — Architecture Medallion

L'architecture retenue s'appuie sur le patron Medallion (aussi appelé Data Lakehouse), combinant la flexibilité d'un Data Lake pour le stockage brut avec la structure et les performances d'un Data Warehouse pour l'analyse. Chaque couche a un rôle précis, une technologie dédiée, et une responsabilité clairement délimitée.

Couche	Nom	Technologie	Rôle
Bronze	RAW	Amazon S3	Stockage brut des données sources sans aucune transformation. Fichiers GTFS, flux JSON d'API, HTML, flux protobuf GTFS-RT.
Silver	STAGING	PostgreSQL + Snowflake staging	Données validées, dédoublées et normalisées. PostgreSQL sert les flux temps réel et l'API. Snowflake staging stocke les données historiques nettoyées.
Gold	MART	Snowflake mart (dbt)	Données agrégées et optimisées pour l'analyse. Tables de faits et de dimensions construites par dbt depuis la couche Silver Snowflake.

Tableau 5 — Architecture Medallion

Ce découpage Bronze / Silver / Gold est un standard de l'industrie data (architecture Medallion). Il garantit que chaque étape de traitement est traçable et réversible : en cas d'erreur de transformation, on peut toujours repartir des données brutes conservées dans la couche Bronze.

a) Justification du choix Amazon S3 (couche Bronze)

Amazon S3 a été retenu comme solution de stockage brut pour les raisons suivantes :

- Scalabilité illimitée : S3 stocke n'importe quel volume de données sans configuration particulière, adapté aux fichiers GTFS volumineux et aux milliers de flux GTFS-RT
- Coût faible : le stockage S3 Standard coûte environ 0,020 €/Go/mois, adapté aux volumes importants de fichiers et de logs de scraping
- Compatibilité universelle : S3 est compatible avec l'ensemble de l'écosystème data (Spark, Airflow, dbt, Snowflake)
- Durabilité : Amazon garantit 99,99% de durabilité des objets stockés

b) Justification du choix PostgreSQL (couche Silver — opérationnel temps réel)

PostgreSQL aura le rôle de couche Silver au côté de Snowflake : il sert de base de données opérationnelle pour les flux temps réel ingérés via Kafka (GTFS-RT) et répond aux requêtes de l'API publique d'EuroMobilityDataHub. Ce rôle nécessite des performances en écriture élevées et une latence faible en lecture, ce que PostgreSQL offre :

- Transactions ACID garantissant l'intégrité lors des mises à jour simultanées (retards, suppressions de trains).
- Support des types JSONB pour stocker les identifiants multi-opérateurs et les coordonnées GPS.
- Indexation avancée (GIN, BRIN) pour les requêtes fréquentes de l'API (recherche par gare, par heure de départ).
- Extensibilité via PostGIS pour les calculs géographiques (distance entre gares, rayon de recherche).

c) Justification du choix Snowflake (Silver staging + Gold mart)

Snowflake intervient à deux niveaux. En Silver, son schéma staging reçoit les données historiques nettoyées depuis S3 via COPY INTO. En Gold, son schéma mart contient les tables analytiques construites par dbt. Cette double utilisation de Snowflake est justifiée par :

- Sa séparation du stockage et du calcul, permettant de scaler les transformations dbt indépendamment du stockage.
- Son intégration native avec dbt, l'outil de transformation utilisé pour promouvoir les données du Silver vers le Gold.
- Ses performances en lecture analytique sur de grands volumes (columnar storage, micro-partitioning).
- Son modèle de facturation à l'usage (pay-per-query), adapté à un projet en croissance.

3.2 Modèle de données

Le défi central d'EuroMobilityDataHub est d'unifier des données issues de formats hétérogènes (GTFS, JSON propriétaire, CSV, protobuf GTFS-RT) en un modèle commun couvrant à la fois le ferroviaire longue distance (SNCF, DB, Trenitalia) et le transport urbain (RATP, Naolib, EMT Madrid). Le modèle retenu s'inspire du standard GTFS tout en l'étendant pour accueillir les spécificités de chaque opérateur et les données temps réel.

a) Tables du modèle unifié

Le modèle est organisé en dimensions (données de référence stables) et en faits (données dynamiques fréquemment mises à jour). Chaque table existe à la fois dans PostgreSQL (pour les besoins opérationnels) et dans Snowflake staging (pour les besoins analytiques).

Table	Type	Description
dim_stations	Dimension	Référentiel de toutes les gares et arrêts : identifiant unifié, nom, pays, coordonnées GPS, type (ferroviaire / urbain), codes opérateurs (JSONB).
dim_operators	Dimension	Liste des opérateurs : nom, pays, type de réseau (rail / urbain), type de source (API, GTFS, scraping).
dim_routes	Dimension	Lignes : identifiant, nom, opérateur, type (TGV, IC, Régional, Métro, Tram, Bus).
fact_trips	Fait dynamique	Trajets planifiés : ligne, gare de départ, gare d'arrivée, date, heure planifiée, opérateur.
fact_fares	Fait dynamique	Tarifs : trajet, classe, type de billet, prix, devise, date de collecte.
fact_realtime	Temps réel	Mises à jour GTFS-RT : identifiant de voyage, retard en secondes, statut, horodatage. Écrit par Kafka, lu par l'API.

Tableau 6 — Tables du modèle de données unifié

b) Règles d'intégrité du modèle

Plusieurs contraintes d'intégrité sont appliquées au niveau de la base de données pour garantir la cohérence du modèle :

- Clés primaires UUID : chaque table possède un identifiant unique généré indépendamment des identifiants sources, qui varient d'un opérateur à l'autre.
- Clés étrangères : les tables de faits référencent obligatoirement les dimensions correspondantes (dim_stations, dim_operators, dim_routes).
- Contraintes NOT NULL : les champs critiques (heure de départ, gare de départ, opérateur) ne peuvent pas être nuls.
- Contraintes CHECK : les prix de fact_fares doivent être strictement positifs ; les retards de fact_realtime sont bornés entre -120 et +1440 minutes.
- Index de performance : créés sur les colonnes les plus filtrées (departure_station_id, scheduled_departure, operator_id) pour optimiser les lectures API.

3.3 PostgreSQL — Base opérationnelle (Silver temps réel)

PostgreSQL héberge la couche Silver opérationnelle. Elle reçoit en continu les mises à jour temps réel écrites par les consommateurs Kafka (flux GTFS-RT de SNCF et DB Urban) et répond aux requêtes de l'API REST publique. Le schéma ci-dessous crée les tables principales avec leurs contraintes d'intégrité (Annexe 6)

3.4 Amazon S3 et Snowflake — Du Bronze au Gold

a) Organisation du Data Lake S3 (couche Bronze)

Les données brutes sont organisées dans S3 selon une arborescence partitionnée par opérateur, type de données et date. Cette organisation permet à Snowflake d'optimiser les lectures par partition (partition pruning), réduisant le volume de données scanné lors du COPY INTO. (Annexe 7)

b) Chargement Bronze → Silver (Snowflake staging)

Le chargement depuis S3 vers Snowflake s'effectue via la commande COPY INTO. Les données atterrissent dans le schéma staging de Snowflake, qui constitue la couche Silver analytique. À ce stade, les données ont été extraites du format brut mais n'ont pas encore subi les transformations métier. (Annexe 8)

c) Promotion Silver → Gold via dbt

Une fois les données chargées dans Snowflake staging (Silver), dbt (data build tool) prend le relais pour construire les tables analytiques du schéma mart (Gold). dbt applique les règles métier, les jointures entre dimensions et faits, et les agrégations nécessaires à l'exposition via l'API analytique. (Annexe 9)

Le schema staging de Snowflake (Silver) est la source de vérité pour dbt. Si une transformation Gold produit un résultat incorrect, on peut la reconstruire depuis le Silver sans recharger depuis S3. Cette séparation garantit la traçabilité et la réversibilité de chaque étape.

3.5 Synthèse — Flux de données bout en bout

L'architecture en trois couches couvre l'intégralité du cycle de vie des données d'EuroMobilityDataHub, depuis la collecte brute jusqu'à l'exposition analytique. Chaque couche a un rôle non redondant :

Couche	Technologie	Données	Flux entrant / sortant
Bronze	Amazon S3	GTFS, JSON, HTML, protobuf	Entrant : Airflow (collecte), Kafka (GTFS-RT). Sortant : Snowflake COPY INTO.
Silver temps réel	PostgreSQL	fact_realtime, dim_stations	Entrant : consommateurs Kafka. Sortant : API REST publique.
Silver analytique	Snowflake staging	dim_stations, fact_trips, fact_fares	Entrant : COPY INTO depuis S3. Sortant : dbt (promotion vers Gold).
Gold	Snowflake mart	Tables analytiques enrichies	Entrant : dbt depuis staging. Sortant : API analytique, dashboards Grafana.

Tableau 7 — Flux de données bout en bout par couche

Ce découpage évite tout court-circuit entre les couches : les données brutes de S3 ne sont jamais exposées directement au Gold. Elles transitent obligatoirement par le Silver, ce qui garantit que chaque enregistrement Gold a été validé, dédoublé et enrichi. PostgreSQL et Snowflake staging jouent des rôles complémentaires dans la couche Silver : le premier gère la basse latence pour le temps réel, le second gère le volume pour l'analytique.

Partie 4 — Transformation des données et processus ETL

Les données collectées arrivent dans tous les formats. Trois fuseaux horaires cohabitent (UTC, CET, WET), les dates changent de forme d'une source à l'autre, les gares portent des noms multilingues, les prix s'expriment en EUR, GBP ou CHF. Impossible, en l'état, de comparer ou d'agréger quoi que ce soit. La transformation devient donc l'étape la plus critique d'EuroMobilityDataHub. Cette partie détaille les outils retenus, les transformations appliquées, puis le processus ETL complet.

4.1 Sélection des technologies de traitement

Trois outils complémentaires seront utilisés pour couvrir l'ensemble des besoins de transformation, des petits volumes aux traitements à grande échelle.

Outil	Usage principal	Avantages	Inconvénients
Python (pandas)	Nettoyage, normalisation, chargement staging	Simple, rapide à développer, large écosystème	Limité aux données tenant en mémoire RAM
Apache Spark (PySpark)	Traitement des gros volumes GTFS et temps réel	Distribué, scalable, traite des To de données	Complexité opérationnelle, temps de démarrage
dbt (data build tool)	Transformations SQL dans Snowflake	Versionné Git, tests intégrés, documentation auto	Uniquement SQL, pas de logique impérative

Tableau 8 — Comparatif des technologies de traitement

Pour le traitement des données, nous adaptons nos outils à la taille des fichiers. Python s'occupe des tâches légères, comme la préparation des données issues du scraping et des API. En revanche, pour les fichiers horaires volumineux (GTFS Static) qui atteignent plusieurs Go, ainsi que pour les flux en temps réel, nous utilisons PySpark pour sa capacité à gérer de gros volumes. Enfin, une fois toutes ces données centralisées dans Snowflake, l'outil dbt prend le relais pour finaliser les calculs et structurer les tableaux de bord analytiques.

4.2 Transformations appliquées aux données

Six types de transformations seront appliqués aux données EuroMobilityDataHub, ce qui couvrira l'ensemble du cycle de nettoyage et de mise en forme.

a) Formatage — Normalisation des fuseaux horaires et formats de date

Chaque opérateur horodate à sa façon. La SNCF renvoie de l'ISO 8601 (UTC+01:00), Deutsche Bahn des timestamps Unix ; les fichiers GTFS, eux, notent les heures en HH:MM:SS. (Annexe 10)

b) Profilage — Analyse de la qualité des données

Avant la transformation, un profilage des données est réalisé pour identifier les anomalies : valeurs manquantes, doublons, outliers tarifaires. Ce profilage est systématisé avec la librairie pandas-profiling (ydata-profiling) et ses résultats sont consignés dans un rapport de qualité. (Annexe 11)

c) Consolidation — Harmonisation des noms de gares

Un même lieu porte parfois plusieurs noms. Tout dépend de l'opérateur : la SNCF affiche «Paris Gare de Lyon», Eurostar écrit «Paris-Lyon», Trenitalia opte pour «Paris Lyon». Pour rapprocher ces variantes, on maintiendra une table de mapping. Chaque nom y renverra à un identifiant unique, assorti d'un libellé normalisé. (Annexe 12)

d) Jointure — Enrichissement des trajets avec les gares

Les trajets remontés par les APIs ne portent que des codes de gare. Chaque opérateur utilise les siens. On joint donc chaque trajet au référentiel `dim_stations`, qui lui apporte un nom normalisé, des coordonnées GPS et un pays. (Annexe 13)

e) Agrégation — Calcul des statistiques tarifaires

Des agrégations sont calculées sur les tarifs pour alimenter les tableaux de bord analytiques : prix moyen par trajet et par classe, évolution des prix, écart entre le tarif le plus bas et le plus élevé pour un même trajet. (Annexe 14)

f) Calcul — Durée de trajet et retard moyen

La transformation ajoute deux métriques calculées. La durée planifiée du trajet, d'abord, exprimée en minutes. Puis le retard moyen constaté sur les 30 derniers jours, trajet par trajet. La couche Gold les stocke directement : aucune requête n'aura à les recalculer. (Annexe 15)

4.3 Processus ETL complet

Toutes les étapes décrites jusqu'ici se rejoignent dans un même pipeline ETL (Extract, Transform, Load). Apache Airflow l'orchestre de bout en bout. Le schéma ci-dessous retrace le flux complet.

a) Vue d'ensemble de la pipeline ETL

Etape	Outil	Action	Destination
Extract	Python / Scrapy / Kafka	Collecte depuis APIs, GTFS, sites web, flux RT	S3 (couche RAW)
Validate	Python (pandas)	Profilage, détection d'anomalies, rapport qualité	Logs Airflow
Transform	PySpark / Python	Formatage, consolidation, jointures, déduplication	PostgreSQL (Staging)
Load staging	SQLAlchemy	Insertion dans PostgreSQL avec contrôle d'intégrité	PostgreSQL (Staging)
Load warehouse	Snowflake COPY INTO	Chargement depuis S3 vers Snowflake	Snowflake (Gold)
Aggregate	dbt	Calcul des métriques, tables analytiques finales	Snowflake (Gold)

Tableau 9 — Vue d'ensemble du pipeline ETL

b) Implémentation du DAG ETL principal

L'ensemble de la pipeline ETL est orchestré par un DAG Airflow central qui enchaîne les étapes dans le bon ordre et gère les dépendances entre tâches. Si une étape échoue, les suivantes ne se lancent pas : on ne risque donc pas un chargement de données incorrectes. (Annexe 16)

c) Outils d'automatisation et d'orchestration

Trois outils assurent l'automatisation et l'orchestration du processus ETL :

- Apache Airflow : orchestre de pipeline ETL batch, planifie les exécutions via des expressions cron, gère les relances automatiques et fournit une interface de supervision.
- Apache Kafka : assure l'ingestion continue des flux temps réel (GTFS-RT) en parallèle de la pipeline batch, sans blocage ni perte de données.
- dbt : gère les transformations SQL dans Snowflake, versionne les modèles de données sous Git, génère automatiquement la documentation et exécute des tests de qualité après chaque run.

Ces trois outils forment un écosystème complémentaire : Airflow orchestre, Kafka streame, dbt transforme. Ensemble, ils couvrent l'intégralité des besoins d'automatisation de la pipeline EuroMobilityDataHub.

4.4 Résultats des transformations

A l'issue du processus ETL, les données transformées présentent les caractéristiques suivantes par rapport aux données sources :

Dimension	Données sources (brutes)	Données transformées
Fuseaux horaires	UTC, CET, WET, heure locale variable	UTC normalisé
Noms de gares	Variantes multiples par opérateur	Identifiant UUID unique + nom normalisé
Devises	EUR, GBP, CHF selon l'opérateur	EUR avec taux de change appliqué
Doublons	~3% de doublons selon la source	0% après déduplication
Valeurs nulles	~10% de champs manquants	Remplacées ou signalées selon règles métier
Formats de date	ISO 8601, Unix, GTFS (HH:MM:SS)	YYYY-MM-DDTHH:MM:SSZ uniforme

Tableau 10 — Comparaison des données avant et après transformation

4.5 Synthèse

Le processus ETL d'EuroMobilityDataHub couvre l'ensemble du cycle de vie des données, depuis la collecte brute jusqu'aux tables analytiques finales dans Snowflake. Les six types de transformations appliquées (formatage, profilage, consolidation, jointure, agrégation, calcul) garantissent la qualité et l'exploitabilité des données. L'orchestration par Airflow, Kafka et dbt assure la reproductibilité et la maintenabilité de la pipeline dans le temps.

La partie suivante aborde la dernière dimension du dossier : la stratégie de sécurisation de l'ensemble de ces données.

Partie 5 — Sécurité des données

Bien que la majorité des données traitées soient publiques (horaires, tarifs), tout n'est pas exposable : il nous faut protéger les données de comportement des utilisateurs de l'API, les clés d'accès aux services tiers, ainsi que les informations techniques sur l'infrastructure.

5.1 Politique de sécurité des données

a) Identification des enjeux et classification des données

Toutes les données du projet ne présentent pas le même niveau de sensibilité. Une classification en trois niveaux a été établie :

Niveau	Type de données	Exemples	Mesures appliquées
PUBLIC	Données ferroviaires ouvertes	Horaires, gares, itinéraires	Aucune restriction d'accès
CONFIDENTIEL	Données techniques internes	Clés API, mots de passe BDD, tokens	Chiffrement, Secrets Manager, accès restreint
SENSIBLE	Données à caractère personnel	Logs d'utilisation de l'API, adresses IP	RGPD, anonymisation, durée de conservation limitée

Tableau 11 — Classification des données par niveau de sensibilité

Le RGPD (Règlement Général sur la Protection des Données) s'applique dès lors que des données permettant d'identifier directement ou indirectement une personne physique sont traitées. Dans EuroMobilityDataHub, les logs d'accès à l'API contenant des adresses IP sont concernés.

b) Évaluation des risques

Une analyse des risques a été conduite selon la méthode EBIOS Risk Manager, identifiant les principales menaces pesant sur la plateforme :

Risque identifié	Probabilité	Impact	Mesure de mitigation
Fuite des clés API tierces (SNCF, DB...)	Moyenne	Elevé	AWS Secrets Manager + rotation automatique
Accès non autorisé à la base PostgreSQL	Faible	Elevé	VPC privé, pare-feu, authentification forte
Injection SQL sur l'API exposée	Moyenne	Elevé	Requêtes préparées, validation des entrées
Perte de données (panne matérielle)	Faible	Moyen	Sauvegardes quotidiennes S3, réplication RDS
Collecte excessive de données personnelles	Faible	Elevé	Anonymisation des logs, minimisation RGPD

Tableau 12 — Analyse des risques (méthode EBIOS Risk Manager)

c) Gestion des droits d'accès selon les rôles

Le principe du moindre privilège est appliqué : chaque composant et chaque utilisateur n'accède qu'aux données strictement nécessaires à sa fonction. Quatre rôles ont été définis :

Rôle	Profil	Droits accordés
admin	Responsable data engineering	Lecture + écriture sur toutes les couches, gestion des utilisateurs
etl_service	Compte de service Airflow	Ecriture uniquement sur S3 RAW et PostgreSQL Staging
analyst	Data analyst, chef de projet	Lecture seule sur Snowflake Gold, aucun accès au Staging
api_consumer	Applications clientes de l'API	Lecture seule sur les endpoints exposés, quotas de requêtes

Tableau 13 — Gestion des droits d'accès par rôle

La gestion de ces rôles est centralisée via AWS IAM (Identity and Access Management) pour les ressources cloud, et via des rôles PostgreSQL natifs pour la base de staging. Chaque accès est journalisé dans AWS CloudTrail. (Visuel Annexe 17)

d) Conformité RGPD

Le RGPD s'applique au projet EuroMobilityDataHub dès lors que des données permettant d'identifier indirectement un utilisateur sont traitées, notamment les adresses IP collectées dans les logs d'accès à l'API. Les obligations suivantes sont respectées :

- La minimisation des données est assurée en ne conservant dans les logs que les informations strictement nécessaires à la détection d'abus. Tous les champs identifiants non indispensables sont supprimés dès la collecte.
- La durée de conservation est limitée à 30 jours pour les logs d'accès à l'API, via une politique de cycle de vie S3 appliquée automatiquement. L'anonymisation des adresses IP est systématique : les deux derniers octets sont masqués avant tout stockage définitif, rendant toute réidentification impossible (méthode recommandée par la CNIL).
- Le droit à l'oubli est pris en compte via un mécanisme de suppression sur demande pour les données utilisateurs éventuellement collectés.

Dans une optique de renforcement de la conformité au RGPD, une alternative européenne a été étudiée : OVHcloud, un hébergeur français dont les données sont stockées exclusivement sur le territoire européen. Cette solution pourrait remplacer AWS dans une version future du projet, en remplaçant S3 par OVHcloud Object Storage, CloudTrail par OVHcloud Logs Data Platform et GuardDuty par Wazuh. Le choix d'AWS pour ce projet repose sur sa maturité technique et son *Free Tier* adapté au contexte d'un projet de fin d'études.

5.2 Architecture de sécurité

a) Zones de sécurité et plan d'adressage

L'architecture de sécurité repose sur un cloisonnement réseau strict organisé en quatre zones, déployées dans un VPC (Virtual Private Cloud) AWS. Chaque zone a un niveau de confiance et des règles d'accès différents. (Annexe

Zone	Sous-réseau (CIDR)	Composants	Accès autorisé
Zone publique (DMZ)	10.0.1.0/24	API Gateway, Load Balancer	Internet -> DMZ uniquement
Zone applicative	10.0.2.0/24	Airflow, Kafka, scripts ETL	DMZ -> Applicative, pas d'accès direct depuis Internet
Zone données (privée)	10.0.3.0/24	PostgreSQL RDS, S3 VPC Endpoint	Applicative -> Données uniquement
Zone administration	10.0.4.0/24	Bastion SSH, outils de monitoring	VPN administrateur uniquement

Tableau 14 — Zones de sécurité et plan d'adressage réseau

Les règles de pare-feu (Security Groups AWS) implémentent le principe de liste blanche : tout trafic est bloqué par défaut, seuls les flux explicitement autorisés sont permis. La communication entre zones est unidirectionnelle : la zone donnée ne peut jamais initier une connexion vers l'extérieur.

b) Flux d'échanges de données

- Internet -> Zone DMZ : les requêtes clientes arrivent sur l'API Gateway via HTTPS (TLS 1.3). Le certificat SSL est géré par AWS Certificate Manager.
- Zone DMZ -> Zone applicative : l'API Gateway transmet les requêtes au service backend via le Load Balancer interne. Ce flux est chiffré avec TLS.
- Zone applicative -> Zone données : les scripts ETL et Airflow accèdent à PostgreSQL via SSL obligatoire (sslmode=require). L'accès à S3 se fait via un VPC Endpoint privé, sans transit par Internet.
- Zone administration -> Toutes zones : les administrateurs se connectent via un VPN WireGuard puis via un hôte Bastion SSH. L'authentification utilise des clés SSH RSA 4096 bits, les mots de passe sont désactivés.

c) Chiffrement des données

Le chiffrement est appliqué à deux niveaux pour protéger les données en transit et au repos

Situation	Standard appliqué	Mise en oeuvre
Données en transit	TLS 1.3	HTTPS sur tous les endpoints, SSL sur connexions PostgreSQL
Données au repos (S3)	AES-256 (SSE-S3)	Chiffrement côté serveur activé sur tous les buckets S3
Données au repos (RDS)	AES-256 (AWS KMS)	Chiffrement des volumes PostgreSQL via AWS KMS
Clés et secrets	AWS Secrets Manager	Rotation automatique des clés API toutes les 30 jours

Tableau 15 — Standards de chiffrement appliqués

Les clés API des opérateurs ferroviaires (SNCF, DB, NS, etc.) seront stockées dans AWS Secrets Manager. Les scripts les récupéreront dynamiquement au moment de leur exécution afin de les injecter dans les environnements des Dockers.

```
import boto3
import json

def get_api_key(secret_name):
    client = boto3.client('secretsmanager', region_name='eu-west-1')
    response = client.get_secret_value(SecretId=secret_name)
    secret = json.loads(response['SecretString'])
    return secret['api_key']

# Utilisation : récupération dynamique, jamais en dur dans le code
SNCF_API_KEY = get_api_key('euroraildatahub/sncf/api-key')
```

d) Surveillance et détection des incidents

Un système de surveillance à plusieurs niveaux sera mis en place pour détecter toute anomalie de sécurité :

- **AWS CloudTrail** : Log chaque action sur les ressources AWS.
- **AWS GuardDuty** : Analyse en continu les logs, ainsi que les VPC Flow Logs et le trafic DNS. Il y repère les comportements suspects : IP malveillante, Brute Force,...
- **Alertes Airflow** : tout échec de tâche ETL déclenche une notification par email à sur la boîte mail d'alerting.
- **Monitoring PostgreSQL** : Pour toutes les connexions échouées, un déclencheur SQL réalisera une alerte pré-configurées.
- **Grafana** : les métriques de sécurité (connexions échouées, volume de requêtes anormal, latence des pipelines) remontent dans un dashboard Grafana dédié.

e) Sauvegarde et plan de reprise

Une stratégie de sauvegarde suivant la règle 3-2-1 : trois copies des données, sur deux supports différents, dont une hors site.

- PostgreSQL RDS : sauvegardes automatiques quotidiennes conservées 7 jours. Snapshots manuels hebdomadaires conservés 30 jours. Réplication multi-AZ pour la haute disponibilité.
- S3 : versioning activé sur tous les buckets. Les objets supprimés accidentellement sont récupérables pendant 30 jours. Les données sont aussi répliquées vers eu-central-1 (Francfort), au cas où toute la région tomberait.
- Objectif de reprise : RTO (Recovery Time Objective) de 4 heures, RPO (Recovery Point Objective) de 24 heures.

5.3 Synthèse de la politique de sécurité

En résumé, la sécurité du projet s'articule autour de la classification des données, du moindre privilège pour les accès, du chiffrement systématique (en transit comme au repos) et de la surveillance via les outils AWS.

Le cloisonnement en zones limite aussi les risques : si une zone est compromise, le reste du système n'est pas exposé. Les procédures de sauvegarde et de reprise assurent la résilience de la plateforme face aux incidents techniques.

Conclusion

Ce dossier présente l'ensemble de la stratégie de collecte, transformation et sécurisation des données du projet EuroMobilityDataHub, une plateforme data unifiée pour l'accès aux informations ferroviaires européennes.

Les cinq parties du dossier couvrent le cycle complet de la donnée : depuis sa collecte via des APIs, flux GTFS, web scraping et requêtes SQL, en passant par son automatisation avec Apache Airflow et Apache Kafka, son stockage dans une architecture en trois couches (S3, PostgreSQL, Snowflake), sa transformation via un processus ETL complet orchestré par dbt, jusqu'à sa sécurisation par chiffrement, gestion des accès et conformité RGPD.

Le choix des technologies s'est fait en fonction des besoins : Kafka pour absorber le temps réel, dbt pour transformer les données de Silver vers Gold, PostgreSQL côté API et Snowflake côté analytique. Pour la sécurité, deux cas de figure : ce qui tourne dans le VPC est protégé par des Security Groups, tandis que Snowflake, hébergé en dehors, est verrouillé via une Network Policy.

Annexes

Les pages suivantes sont dédiées aux annexes du document et contiennent les informations complémentaires associées.

Annexe 1 :

```
from google.transit import gtfs_realtime_pb2
import requests

# Exemple avec le flux de l'ÖBB (Autriche) via la Mobility Database
url = 'https://api.example.com/gtfs-rt'
response = requests.get(url)

feed = gtfs_realtime_pb2.FeedMessage()
feed.ParseFromString(response.content)

for entity in feed.entity:
    if entity.HasField('trip_update'):
        print(f"Train ID: {entity.trip_update.trip.trip_id}")
        print(f"Retard: {entity.trip_update.stop_time_update[0].departure.delay} secondes")
```

Annexe 2 :

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto('https://www.renfe.com/es/fr')
    # Remplissage du formulaire et extraction des tarifs
    tarifs = page.query_selector_all('.fare-price')
    for t in tarifs:
        print(t.inner_text())
```

Annexe 3 :

```
INSERT INTO staging.fact_departures_consolidated
SELECT DISTINCT ON (trip_id, scheduled_departure)
    trip_id,
    departure_station_id,
    scheduled_departure,
    real_departure,
    source,
    collected_at
FROM staging.fact_departures_raw
WHERE collected_at >= NOW() - INTERVAL '24 hours'
ORDER BY trip_id, scheduled_departure, collected_at DESC;
```

Annexe 4 :

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin

BASE_URL = 'https://www.example-operator.com/gares/'
visited, to_visit = set(), [BASE_URL]
new_stations = []

while to_visit:
    url = to_visit.pop()
    if url in visited:
        continue
```

```

visited.add(url)
soup = BeautifulSoup(requests.get(url, timeout=30).content, 'html.parser')
# Extraction des gares de la page
for station in soup.select('.station-name'):
    new_stations.append(station.get_text(strip=True))
# Decouverte des liens internes vers d'autres pages de gares
for link in soup.select('a[href^="/gares/"]'):
    to_visit.append(urljoin(BASE_URL, link['href']))

print(f'{len(new_stations)} gares decouvertes')

```

Annexe 5 :

```

from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
import requests, zipfile, io

GTFS_SOURCES = {
    'snCF':
        'https://eu.ftp.opendatasoft.com/snCF/gtfs/export-ter-gtfs-last.zip',
    'trenitalia':
        'https://www.trenitalia.com/content/dam/tcom/gtfs/trenitalia_gtfs.zip',
    'obb':
        'https://data.oebb.at/oebb?dataset=gtfs&format=zip',
}

def download_gtfs(operator, url):
    r = requests.get(url, timeout=60)
    z = zipfile.ZipFile(io.BytesIO(r.content))
    z.extractall(f'/data/gtfs/{operator}/')
    print(f'GTFS {operator} telecharge avec succes')

with DAG(
    dag_id='collect_gtfs_static',
    schedule_interval='0 3 * * 1',
    start_date=datetime(2025, 9, 1),
    catchup=False,
    default_args={'retries': 3, 'retry_delay': timedelta(minutes=10)}
) as dag:
    for operator, url in GTFS_SOURCES.items():
        PythonOperator(
            task_id=f'download_{operator}',
            python_callable=download_gtfs,
            op_kwargs={'operator': operator, 'url': url}
        )

```

Annexe 6 :

```

-- Schema operationnel EuroMobilityDataHub
CREATE SCHEMA IF NOT EXISTS operational;

-- Referentiel des gares et arrêts (rail + urbain)
CREATE TABLE operational.dim_stations (
    station_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    unified_name       VARCHAR(200) NOT NULL,
    country_code       CHAR(2) NOT NULL,
    stop_type          VARCHAR(10) CHECK (stop_type IN ('rail','urban')) NOT NULL,
    network_type       VARCHAR(10) CHECK (network_type IN
('metro','tram','bus','rail')),
    latitude           DECIMAL(9,6),
    longitude          DECIMAL(9,6),
    operator_codes     JSONB,
    created_at         TIMESTAMP DEFAULT NOW()
);

-- Operateurs (rail et urbain)

```

```

CREATE TABLE operational.dim_operators (
  operator_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name         VARCHAR(100) NOT NULL,
  country_code CHAR(2) NOT NULL,
  network_type VARCHAR(10) CHECK (network_type IN ('rail','urban')) NOT NULL,
  source_type  VARCHAR(20) CHECK (source_type IN ('api','gtfs','scraping'))
);

-- Mises a jour temps reel (ecrit par Kafka)
CREATE TABLE operational.fact_realtime (
  update_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trip_id    VARCHAR(100) NOT NULL,
  delay_sec  INTEGER CHECK (delay_sec BETWEEN -7200 AND 86400),
  status     VARCHAR(20) CHECK (status IN ('on_time','delayed','cancelled')),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Index pour l'API (requetes par gare et par heure)
CREATE INDEX idx_stations_country ON operational.dim_stations(country_code,
stop_type);
CREATE INDEX idx_realtime_trip ON operational.fact_realtime(trip_id, updated_at
DESC);

```

Annexe 7 :

```

s3://euromobilitydatahub-raw/
  gtfs/
    sncf/year=2026/month=06/day=09/sncf_gtfs.zip
    trenitalia/year=2026/month=06/day=09/trenitalia_gtfs.zip
    ratp/year=2026/month=06/day=09/ratp_gtfs.zip
    naolib/year=2026/month=06/day=09/naolib_gtfs.zip
  realtime/
    sncf/year=2026/month=06/day=09/hour=14/feed_1234567890.pb
    ratp/year=2026/month=06/day=09/hour=14/feed_9876543210.pb
  fares/
    renfe/year=2026/month=06/day=09/renfe_fares.json
    db/year=2026/month=06/day=09/db_fares.json

```

Annexe 8 :

```

-- Stage S3 dans Snowflake
CREATE OR REPLACE STAGE euromobilitydatahub_stage
  URL = 's3://euromobilitydatahub-raw/'
  CREDENTIALS = (AWS_KEY_ID = '...' AWS_SECRET_KEY = '...');

-- Chargement Bronze -> Silver (schema staging)
COPY INTO staging.dim_stations
  (unified_name, country_code, stop_type, latitude, longitude)
FROM (
  SELECT $1, $2, $3, $4::FLOAT, $5::FLOAT
  FROM @euromobilitydatahub_stage/gtfs/sncf/
)
FILE_FORMAT = (TYPE = 'CSV' SKIP_HEADER = 1)
ON_ERROR = 'CONTINUE';

-- Verification du chargement Silver
SELECT COUNT(*), MAX(created_at) FROM staging.dim_stations;

```


Annexe 9 :

```
-- models/mart/dim_stations.sql (modele dbt Gold)
{{ config(schema='mart', materialized='table') }}

SELECT
    station_id,
    unified_name,
    country_code,
    stop_type,
    network_type,
    latitude,
    longitude,
    operator_codes,
    -- Enrichissement metier : calcul de la region
    CASE country_code
        WHEN 'FR' THEN 'Western Europe'
        WHEN 'DE' THEN 'Central Europe'
        WHEN 'IT' THEN 'Southern Europe'
        ELSE 'Other'
    END AS region,
    CURRENT_TIMESTAMP AS gold_loaded_at
FROM {{ source('staging', 'dim_stations') }}
WHERE unified_name IS NOT NULL
    AND latitude IS NOT NULL
```

Annexe 10 :

```
import pandas as pd
from datetime import datetime, timezone

def normalize_datetime(raw_time, source_tz='Europe/Paris'):
    # Convertit n'importe quel format d'heure en UTC
    if isinstance(raw_time, int): # Timestamp Unix (Deutsche Bahn)
        dt = datetime.fromtimestamp(raw_time, tz=timezone.utc)
    elif isinstance(raw_time, str): # ISO 8601 (SNCF)
        dt = pd.to_datetime(raw_time).tz_convert('UTC')
    else:
        raise ValueError(f'Format non reconnu : {type(raw_time)}')
    return dt.strftime('%Y-%m-%dT%H:%M:%SZ')

# Application sur le DataFrame SNCF
df['scheduled_departure'] = df['departure_time'].apply(
    lambda t: normalize_datetime(t, source_tz='Europe/Paris')
)
```

Annexe 11 :

```
import pandas as pd

def profile_dataset(df, source_name):
    report = {
        'source': source_name,
        'total_rows': len(df),
        'null_counts': df.isnull().sum().to_dict(),
        'duplicate_rows': df.duplicated().sum(),
        'price_outliers': len(df[df['price'] > df['price'].quantile(0.99)])
    }
    print(f'Profil {source_name}: {report}')
    return report

# Exemple : profilage des tarifs Renfe
profile_dataset(df_renfe_fares, 'renfe_fares')
# Résultat : {'total_rows': 4820, 'duplicate_rows': 12, 'price_outliers': 3}
```

Annexe 12 :

```
# Table de correspondance des noms de gares
station_mapping = {
    'paris gare de lyon': 'station_uuid_paris_lyon',
    'paris-lyon':         'station_uuid_paris_lyon',
    'paris lyon':         'station_uuid_paris_lyon',
    'gare de lyon':       'station_uuid_paris_lyon',
}

def consolidate_station(raw_name):
    key = raw_name.lower().strip()
    return station_mapping.get(key, None)

df['station_id'] = df['raw_station_name'].apply(consolidate_station)

# Vérification : stations non mappées
unmapped = df[df['station_id'].isnull()]['raw_station_name'].unique()
print(f'Gares non mappees : {len(unmapped)} -> {unmapped[:5]}')
```

Annexe 13 :

```
-- Jointure SQL dans PostgreSQL (couche Staging)
INSERT INTO staging.fact_trips_enriched
SELECT
    t.trip_id,
    t.scheduled_departure,
    t.scheduled_arrival,
    dep.unified_name    AS departure_station_name,
    dep.country_code   AS departure_country,
    dep.latitude        AS departure_lat,
    dep.longitude       AS departure_lon,
    arr.unified_name    AS arrival_station_name,
    arr.country_code    AS arrival_country
FROM staging.fact_trips t
JOIN staging.dim_stations dep ON t.departure_station_id = dep.station_id
JOIN staging.dim_stations arr ON t.arrival_station_id = arr.station_id;
```

Annexe 14 :

```
-- Modèle dbt : agrégation des tarifs par trajet (couche Gold, Snowflake)
SELECT
    trip_id,
    class,
    MIN(price)                AS price_min,
    MAX(price)                AS price_max,
    ROUND(AVG(price), 2)      AS price_avg,
    COUNT(*)                  AS fare_count,
    MAX(price) - MIN(price)    AS price_spread,
    MAX(collected_at)         AS last_updated
FROM {{ ref('fact_fares') }}
GROUP BY trip_id, class
```

Annexe 15 :

```
-- Calcul de la durée et du retard moyen
SELECT
    t.trip_id,
    DATEDIFF('minute',
            t.scheduled_departure,
            t.scheduled_arrival) AS planned_duration_min,
    ROUND(AVG(r.delay_seconds) / 60, 1) AS avg_delay_min_30d
FROM {{ ref('fact_trips') }} t
LEFT JOIN {{ ref('fact_realtime') }} r
    ON t.trip_id = r.trip_id
    AND r.updated_at >= DATEADD('day', -30, CURRENT_DATE)
GROUP BY t.trip_id, t.scheduled_departure, t.scheduled_arrival
```

Annexe 16 :

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.snowflake.operators.snowflake import SnowflakeOperator
from datetime import datetime, timedelta

with DAG(
    dag_id='etl_pipeline_euroraildatahub',
    schedule_interval='0 4 * * *', # Tous les jours à 4h du matin
    start_date=datetime(2025, 9, 1),
    catchup=False,
    default_args={'retries': 2, 'retry_delay': timedelta(minutes=5)}
) as dag:

    extract = PythonOperator(
        task_id='extract_all_sources',
        python_callable=extract_all_sources,
    )

    validate = PythonOperator(
        task_id='validate_and_profile',
        python_callable=validate_and_profile,
    )

    transform = PythonOperator(
        task_id='transform_and_normalize',
        python_callable=transform_and_normalize,
    )

    load_staging = PythonOperator(
        task_id='load_to_postgresql',
        python_callable=load_to_postgresql,
    )

    load_warehouse = SnowflakeOperator(
        task_id='copy_to_snowflake',
        sql='COPY INTO gold.fact_trips FROM @euroraildatahub_stage;',
        snowflake_conn_id='snowflake_default',
    )

    dbt_run = PythonOperator(
        task_id='dbt_run_models',
        python_callable=run_dbt_models,
    )

    # Définition de l'ordre d'exécution
    extract >> validate >> transform >> load_staging >> load_warehouse >> dbt_run
```

Annexe 17 :

Architecture de sécurité de la plateforme centralisée de données ferroviaires européennes.

